

AntiFLipper: A Secure and Efficient Defense Against Label-Flipping Attacks in Federated Learning

Aashnan Rahman¹, Abid Hasan², Sherajul Arifin³, Faisal Haque Bappy⁴, Tahrim Hossain⁵,
Tariqul Islam⁶, Abu Raihan Mostofa Kamal⁷, and Md. Azam Hossain⁸
^{1,2,7,8} Islamic University of Technology, Bangladesh.

³ United International University, Bangladesh.

^{4,5,6} Syracuse University, United States.

Email: {aashnanrahman@iut-dhaka, abidhasan32@iut-dhaka, sherajul@cse.uiu.ac.bd,
fbappy@syr, mhossa22@syr, mtislam@syr, azam@iut-dhaka, raihan.kamal@iut-dhaka}.edu

Abstract—Federated learning (FL) enables privacy-preserving model training by keeping data decentralized. However, it remains vulnerable to label-flipping attacks, where malicious clients manipulate labels to poison the global model. Despite their simplicity, these attacks can severely degrade model performance, and defending against them remains challenging. We introduce AntiFLipper, a novel and computationally efficient defense against multi-class label-flipping attacks in FL. **The attacker’s capability is strictly limited to local data manipulation (label-flipping), without access to model updates or server components.** Unlike existing methods that ensure security at the cost of high computational overhead, AntiFLipper employs a novel **client-side evaluation strategy computing accuracy locally and sharing only a scalar score with the server**, significantly reducing the central server’s burden during aggregation and making the approach suitable for edge networks. Comprehensive empirical evaluations across multiple datasets under different distributions demonstrate that AntiFLipper achieves accuracy comparable to state-of-the-art defenses while requiring substantially fewer computational resources on the server side. By balancing security and efficiency, AntiFLipper addresses a critical gap in existing defenses, making it particularly suitable for resource-constrained FL deployments (e.g., IoT) where both model integrity and operational efficiency are essential.

Index Terms—Federated Learning, Label Flipping Attack, Malicious Node Detection, Attack Mitigation, IoT, Edge Networks

I. INTRODUCTION

Federated learning (FL) has emerged as a prominent distributed learning paradigm for communication networks and edge computing systems, enabling geographically dispersed clients such as mobile devices, IoT sensors, and edge devices to collaboratively train a global model without sharing raw data [1], [2]. By exchanging only model updates, FL satisfies the strict privacy requirements of applications in healthcare, finance, and industrial IoT. However, the decentralized nature of FL exposes it to data poisoning attacks [3], [4], among which *label-flipping attacks* are particularly concerning for networked deployments. In this attack, malicious clients deliberately mislabel their local training data before submitting model updates. Critically, the attacker’s capability is strictly limited to this local label manipulation, they cannot modify model updates, intercept communications, or access any server or peer data. Despite this simplicity, even a small fraction of

compromised clients can cause substantial degradation of the global model [5], [6], posing a direct threat to the reliability of FL-enabled distributed systems [7].

Existing defenses typically rely on server-side anomaly detection or robust aggregation, requiring the server to inspect, cluster, or compare all client updates [8], [9], [10]. This introduces significant computational overhead at the aggregation server and, in some cases, requires the server to maintain a clean labeled reference dataset [11]—a privacy concern that contradicts FL’s data-minimization principles. Lightweight defenses, on the other hand, often fail under non-IID data distributions prevalent in heterogeneous wireless networks [12].

To address these limitations, we propose AntiFLipper, a defense that shifts detection to the client side through local accuracy evaluation, avoiding server-side model inspection entirely. Unlike approaches that depend on server-held reference data or expensive cryptographic mechanisms, AntiFLipper infers trust from relative client performance deviations, making it well-suited for resource-constrained edge and IoT deployments.

The core contributions of this paper are:

- **Attack Detection & Mitigation:** We propose AntiFLipper, a novel trust-based weighted aggregation method that detects and mitigates multi-class label-flipping attacks by identifying and excluding malicious clients, even in dynamic attack scenarios.
- **Accuracy and Efficiency:** AntiFLipper matches or exceeds state-of-the-art defenses in accuracy while achieving significantly lower aggregation time by avoiding server-side model update inspection.
- **Improved Scalability:** By shifting detection to client devices, AntiFLipper minimizes server burden, improving scalability in large-scale decentralized networks.

The remainder of the paper is structured as follows. In Section II, we describe the problem statement and our research objectives. In Section III, we review existing work related to the detection and defense against label-flipping attacks by malicious clients in federated learning scenarios. In Section IV, we present AntiFLipper in detail. In Section V we present system implementation and performance evaluation. In Section VI, we provide a security analysis, discussing the robustness of our approach in mitigating malicious attacks, including its be-

havior under adaptive and colluding adversaries. In Section VII, we analyze the computational and communication overhead introduced by client-side evaluation. Section VIII discusses the limitations of our approach and threats to validity. Finally, Section IX summarizes the key findings and concludes the paper.

II. PROBLEM STATEMENT

A. Label Flipping Attack

Let $C = \{c_1, c_2, \dots, c_n\}$ be the set of clients participating in the federated learning process. Each client $c_i \in C$ possesses a local dataset $D_i = \{(x_k, y_k)\}_{k=1}^{m_i}$, where $x_k \in \mathcal{X}$ is the input feature and $y_k \in \mathcal{Y}$ is the true class label. In our threat model, the adversary is strictly limited to manipulating its local data (i.e., label-flipping) and has no capability beyond this scope. In a label-flipping attack, the adversary modifies this dataset into a corrupted version $D'_i = \{(x_k, y'_k)\}_{k=1}^{m_i}$, such that $y'_k \neq y_k$ for a substantial subset of the data [13]. These poisoned labels cause the local model m'_i , trained on D'_i , to encode incorrect decision boundaries. A common variant is the *targeted label-flipping attack*, where the adversary selects a source class $s \in \mathcal{Y}$ and a target class $\tau \in \mathcal{Y}$, applying:

$$y'_k = \begin{cases} \tau & \text{if } y_k = s, \\ y_k & \text{otherwise.} \end{cases}$$

The malicious client then trains locally and submits the resulting model m'_i to the server. Over time, the aggregation of poisoned updates subtly shifts the global model's decision boundaries toward the adversary's target class [11], relying solely on local label manipulation without requiring server access or deviating from the standard FL protocol.

B. Threat Model Boundary and Adaptive Adversaries

Our primary threat model, consistent with the label-flipping literature [6], [14], restricts the adversary to local data manipulation: the attacker cannot tamper with model updates, the evaluation procedure, or server-side computation. This scoping is deliberate, as it isolates the specific and widely-studied threat of label corruption from the broader and largely orthogonal problem of Byzantine gradient manipulation, which is already addressed by a separate line of robust-aggregation work [9], [10], [15]. We nonetheless discuss two natural extensions of the adversary's capabilities that a reviewer or practitioner may reasonably ask about.

Falsified accuracy reports. A strictly stronger adversary could additionally misreport its local accuracy acc_{ij} instead of, or in addition to, flipping labels, attempting to mimic acc_{avg} and evade the trust penalty. This capability lies outside our stated threat model (Section II), since it requires the attacker to tamper with the evaluation/reporting pipeline rather than only the local dataset. It is nonetheless a realistic concern in deployments without secure client attestation. A natural mitigation is for the server to occasionally issue a small *challenge set*, a fixed, previously unseen mini-batch distributed identically to all active clients, and compare each client's self-reported

accuracy against the trust-weighted aggregate behavior implied by the client's submitted model update on that same batch; a large mismatch between reported and re-derived accuracy is itself an anomaly signal. We view a full protocol and empirical evaluation of this extension, e.g., via lightweight zero-knowledge proofs of correct local evaluation [16] or commit-then-reveal accuracy reporting, as important future work rather than a claim made by the present evaluation.

Adaptive and colluding label-flippers. A more sophisticated adversary could perform a *partial flip* (poisoning only a fraction of local labels) or coordinate with other malicious clients so that each individual client's accuracy deviation stays just inside the trust threshold τ_{thresh} , trading detectability for reduced attack impact. Because the flag-counter mechanism (M4) already requires *persistent*, multi-round deviation before exclusion, a partial-flip strategy that stays under τ_{thresh} correspondingly bounds the damage that client can inflict on the aggregate, an attacker who wants to remain undetected is, by construction of the trust update rule, also limited in how much it can corrupt M_{i+1} per round. Formalizing this accuracy-impact trade-off, and empirically stress-testing partial-flip and colluding strategies explicitly designed to evade average-based detection, is an important direction we leave to future work (Section VIII); our current Lemma-1/Lemma-2 analysis (Section VI) assumes the constant, full-flip adversary evaluated in Section V.

C. Research Objectives

Existing defenses against label-flipping attacks either rely on centralized anomaly detection at the server, which is computationally expensive, unscalable, and burdensome for resource-constrained environments, or require clients to share portions of their local data, contradicting the privacy-preserving principles of FL.

To address these challenges, our research pursues the following three core objectives: i) *design a decentralized aggregation method* that accurately detects and mitigates label-flipping attacks; ii) *maintain high model performance* and reduce aggregation time even in the presence of adversarial clients, ensuring both robustness and efficiency; and iii) *shift the majority of computations to the client side* and rigorously analyze the trade-offs in system efficiency, scalability, and resilience under adversarial conditions. Together, these objectives aim to build a trustworthy federated learning framework that resists adversarial manipulation while preserving client privacy and enabling scalable deployment.

III. RELATED WORK

A. Label Flipping-Specific Defenses

Several approaches have tackled label-flipping attacks in federated learning. Tolpegin et al. [6] demonstrated the feasibility of such attacks and proposed a Principal Component Analysis based mechanism to detect malicious clients, but their method lacks generalization guarantees and performs poorly in non-IID settings. Fung et al. [12] introduced FoolsGold, leveraging cosine similarity between gradient updates. While effective

against sybil-style poisoning, it suffers from false positives in non-IID scenarios and incurs high computational cost.

Nguyen et al. [8] proposed FLAME, combining HDBSCAN clustering, differential privacy, and noise addition. It preserves model utility but requires careful tuning and introduces substantial overhead. Jebreel et al. [14] developed LFighter, reweighting client updates using feature-level centroid deviation. It improves robustness but its effectiveness declines with data heterogeneity.

Two recent works directly target label-flipping in non-IID FL settings and warrant closer examination. Li et al. [17] proposed an Honest Score Client Selection Scheme that evaluates clients against a clean server-held dataset. While effective in non-IID conditions, this approach requires the server to maintain labeled reference data, introducing a privacy concern as it contradicts the core FL principle that raw and sensitive client data should remain local. Shen et al. [16] introduced a privacy-preserving defense using cryptographic mechanisms. While offering stronger privacy guarantees, the method incurs significant computational overhead due to encryption and secure aggregation, raising scalability concerns for large client populations and resource-constrained edge devices. Notably, both works primarily validate their performance on simpler datasets such as MNIST, without demonstrating effectiveness on more complex datasets like CIFAR-10, raising concerns about generalizability.

More recent label-flipping-specific defenses have continued to explore client-side and behavior-based detection. Jiang et al. [18] proposed MCDFL, which recovers a latent feature-space distribution to estimate per-client data quality, requiring the server to train and maintain a generative estimator, in contrast to AntiFLipper’s single scalar report. Ovi et al. [19] used prediction-confidence statistics to flag flipped-label contributions, while Li et al. [20] (FLGT) and a concurrent dual-granularity distribution-alignment defense [21] guide server-side trust using richer per-client statistics (confidence margins or VAE-derived feature/label distribution signatures) rather than a single accuracy scalar, trading AntiFLipper’s minimal communication footprint for finer-grained but more expensive detection signals. Abroshan [22] instead monitors global-model output behavior (prediction entropy, margins) in an attack-agnostic manner, complementary to AntiFLipper’s client-reported evaluation. Relative to this recent line of work, AntiFLipper’s distinguishing design choice remains that clients compute and report only a single scalar accuracy value, without exposing feature statistics, confidence vectors, or requiring a generative model at the server.

We note that the accuracy-deviation principle underlying AntiFLipper is conceptually related to prior trust- and validation-score-based defenses such as FLTrust [11], which bootstraps trust from a server-held root dataset, and general Byzantine-robust aggregation rules such as Multi-Krum [10], Bulyan [15], and the certified-robustness framework CRFL [23], which operate directly on model updates rather than client-reported evaluation scalars. AntiFLipper’s novelty relative to this broader family lies specifically in (i) requiring no server-held refer-

ence dataset (unlike FLTrust and HSCS [17]), (ii) requiring no inspection of model updates or gradients (unlike Multi-Krum, Bulyan, FLAME, and CRFL), and (iii) transmitting a single scalar rather than a feature or confidence vector (unlike MCDFL [18] and FLGT [20]). We have not empirically benchmarked against Bulyan, CRFL, or FLTrust in this version of the paper, as they target the broader Byzantine/backdoor threat model rather than label-flipping specifically and require additional experimentation; we discuss this scoping decision further in Section VIII.

B. Native Federated Learning Defenses

While not specifically tailored to counter label-flipping attacks, several robust aggregation methods have been studied in this context. FedAvg[1], the popular aggregation mechanism in federated learning, lacks robustness against adversarial updates and suffers significant accuracy degradation under targeted label-flipping. To improve resilience, Yin et al. [9] introduced coordinate-wise Median and Trimmed Mean strategies, demonstrating moderate robustness in IID settings but degraded performance in non-IID scenarios. Multi-Krum [10] uses distance-based filtering, assuming an IID setting and a known number of adversaries; its performance deteriorates with data heterogeneity and its quadratic complexity limits scalability.

In contrast, our proposed method, AntiFLipper, is designed without relying on data distribution assumptions or prior adversary knowledge. It provides a principled aggregation framework that generalizes well across non-IID environments and varying model architectures. Notably, AntiFLipper significantly reduces computational overhead while maintaining high model accuracy across all evaluated scenarios, making it a practical and scalable solution for real-world federated learning applications where robustness and efficiency are critical.

IV. DETAILED DESIGN

AntiFLipper defends federated learning against label-flipping attacks via a lightweight, client-side accuracy evaluation mechanism. The core intuition is that a global model trained mostly on honest data will exhibit markedly lower accuracy on a client’s poisoned data because flipped labels conflict with the learned representations. The client side mechanism independently evaluates the current global model on a subset of the client’s private data and reports only a scalar accuracy to the server, transmitting neither raw data nor gradients and thus preserving privacy. From these scalars, the server maintains a per-client trust score, progressively down-weighting and eventually excluding clients with persistently anomalous accuracy. The method requires no explicit knowledge of the data distribution, it relies on *relative* performance deviations across clients, tolerating transient drops caused by natural data heterogeneity while penalizing consistently anomalous behaviour. Since attackers are restricted to label manipulation, they cannot affect the local evaluation process or any aspect beyond label corruption of training data. By deliberately avoiding server-side model-update inspection, the dominant overhead in competing methods, the framework keeps

aggregation lightweight and scalable for resource-constrained edge and IoT deployments.

The AntiFLipper framework integrates five interdependent modular components as shown in Figure 1. Each module

sys_arc

Fig. 1: Workflow of the AntiFLipper Framework

operates within the standard FL loop, enhancing robustness against label-flipping attacks without altering the fundamental decentralized execution model.

A. M1: Local Evaluation Feedback Module

At the beginning of each communication round, the server distributes the global model M_i to all participating clients not in the malicious list β (Algorithm 1, lines 2–3). Each client-side evaluation mechanism then evaluates M_i on a subset of the client’s local training dataset $data_{ij}$, computing a local accuracy score acc_{ij} . Only this scalar is returned to the server, no raw data is transmitted. Clients with poisoned data yield lower global model accuracy, as the model trained on honest data performs poorly on flipped labels, producing the detection signal. Following evaluation, each client performs local training (Algorithm 1, line 5) to generate model m_{ij} and returns both the model and acc_{ij} to the server.

B. M2: Trust Inference Module

Upon receiving the results, the server computes the average accuracy across all non-malicious clients (i.e., excluding clients identified as malicious up to the current round) (Algorithm 2, line 1). The trust score τ_j , initialized uniformly as $1/N$ where N is the total number of clients, is then updated for each client j based on the squared deviation from the computed average: $\tau_j \leftarrow \tau_j \pm \eta \cdot (acc_{avg} - acc_{ij})^2$. Here, $sqDeviation() = (acc_{ij} - acc_{avg})^2$; $penalizeTrust() = \tau_j - \eta \cdot d$; $rewardTrust() = \tau_j + \eta \cdot d$; and $normalizeTrust()$ rescales trust into $[0, 1]$ excluding malicious clients. The sign is negative if $acc_{ij} < acc_{avg}$ (lines 4–5) and positive otherwise (lines 6–7). This quadratic penalty function penalizes deviations from the average more strongly than a linear scheme, addressing

Algorithm 1: AntiFLipper()

Input : Number of clients n , client list $clients$, local datasets $data$, trust learning rate η , trust threshold $\tau_{threshold}$, flag threshold cnt_{max}

Output: Global model M_{i+1} , malicious clients list β

```

1  $(M_0, \tau, cnt, \beta) \leftarrow \text{Initialize}()$ 
2 for  $i \leftarrow 1$  to  $totalRounds$  do
3    $\text{DistributeModel}(M_i, clients, \beta)$ 
4   foreach  $j \in clients$  and  $j \notin \beta$  do
5      $(m_{ij}, acc_{ij}) \leftarrow \text{LocTrain}(M_i, j, data_{ij})$ 
6    $(\tau, cnt, \beta) \leftarrow$ 
      $\text{MalNodeFilter}(clients, acc_{ij}, \tau, cnt, \beta)$ 
7    $M_{i+1} \leftarrow \text{GlobModelAgg}(m, \tau, clients, \beta)$ 
8 return  $M_{i+1}, \beta$ 
```

the issue where small but persistent differences could accumulate disproportionately over multiple rounds under a linear penalty formulation.

C. M3: Behavior Profiling Module

The server analyzes accuracy and trust scores over multiple rounds to build a behavior profile per client via the flag counter cnt_j (Algorithm 2, lines 8–11), tracking how many rounds a client’s trust has fallen below threshold. AntiFLipper makes no assumptions about data distribution or non-IID structure and uses relative performance deviations across clients for detection. It tolerates occasional accuracy drops from natural heterogeneity, penalizing only persistent anomalies, enabling robust detection without distribution-aware mechanisms.

When malicious clients are few or their influence is negligible, they may not be immediately flagged, as their impact is insufficient to degrade the global model. Since the objective is to prevent meaningful model corruption, such low-impact cases are not critical to the defense goal.

D. M4: Anomaly-Guided Filtering Module

To avoid overreacting to transient accuracy drops, each client’s flag counter cnt_j tracks underperformance rounds. A client with trust score τ_j below $\tau_{thresh} = \frac{1}{kN}$ (a scalar threshold inversely proportional to N , the total number of initial clients, and adjustable via k to control detection speed) is flagged (Algorithm 2, lines 8–9). After cnt_{max} flags, a client is deemed malicious and added to list β (lines 10–11). Flagged clients ($\tau_j < \tau_{thresh}$) have their updates excluded from the next round’s aggregation to prevent model contamination. Malicious clients are excluded from participation in future rounds. Trust values are normalized to $[0, 1]$ (line 12). This balances tolerance for noisy data with protection against adversarial behavior.

The overall malicious node filtering process is implemented in Algorithm 2, which is invoked from the main AntiFLipper algorithm (Algorithm 1, line 6) during each communication round, before global model aggregation.

E. M5: Weighted Aggregation Module

After malicious clients have been filtered out, the remaining client models are aggregated using a weighted average (Algo-

Algorithm 2: MalNodeFilter()

Input : Clients $clients$, Accuracies acc_{ij} , Trust values τ , Flags cnt , Malicious list β

Output: Updated τ , cnt , β

```
1  $acc_{avg} \leftarrow \text{getAvgAccuracy}(clients \setminus \beta)$ 
2 foreach  $j \in clients$  and  $j \notin \beta$  do
3    $d \leftarrow \text{sqDeviation}(acc_{ij}, acc_{avg})$ 
4   if  $acc_{ij} < acc_{avg}$  then
5      $\tau_j \leftarrow \text{penalizeTrust}(\tau_j, d, \eta)$ 
6   else
7      $\tau_j \leftarrow \text{rewardTrust}(\tau_j, d, \eta)$ 
8   if  $\tau_j \leq \tau_{thresh}$  then
9      $cnt_j \leftarrow cnt_j + 1$ 
10    if  $cnt_j \geq cnt_{max}$  then
11       $\text{add } j \text{ to } \beta$ 
12  $\tau \leftarrow \text{normalizeTrust}(\tau, \beta)$ 
13 return  $\tau$ ,  $cnt$ ,  $\beta$ 
```

rithm 1, line 7). The weight assigned to each client’s model is proportional to its normalized trust score. Formally, the aggregated model M_{i+1} is computed as: $M_{i+1} = \sum_{j \in \text{Honest}} \frac{\tau_j}{W} \cdot m_{ij}$, where $W = \sum_{j \in \text{Honest}} \tau_j$. This ensures that reliable clients have a greater impact on the evolution of the global model, reinforcing the positive feedback loop while suppressing the influence of detected advisories.

V. IMPLEMENTATION AND EVALUATION

We developed AntiFlipper¹ in Python and evaluated it under label-flipping attacks against established defenses, including FedAvg [1], Median, Trimmed Mean [9], FoolsGold [12], Tolpegin [6], MKRUM [10], FLAME [8], and LFighter [14]. Experiments were implemented in PyTorch on a 12th Gen Intel Core i9-12900K CPU, 128GB RAM, and NVIDIA RTX 3090 GPU running Ubuntu 22.04.4 LTS. We evaluated on two benchmark datasets: MNIST70K [24], comprising 70,000 grayscale digit images (60K train, 10K test), and CIFAR-10 [25], containing 60,000 color images across 10 classes (50K train, 10K test). MNIST models use a two-layer CNN followed by two fully connected layers, while CIFAR-10 models are based on ResNet18 [26] and ShuffleNetV2 [27], each ending with a single fully connected layer.

Experiments were conducted under both IID and non-IID settings. For MNIST, we simulated 100 clients, all active every round, under an IID random-uniform split and, separately, under Dirichlet [28] non-IID partitioning ($\alpha = 1$), running for 200 communication rounds with 3 local epochs, batch size 64, and SGD (lr=0.001, momentum=0.9). For CIFAR-10, we used 100 clients (20 active per round) under IID and 20 clients (all active per round) under non-IID ($\alpha = 1$), trained over 100 rounds with 3 local epochs, batch size 32, and SGD (lr=0.01, momentum=0.9). [AUTHOR TO CONFIRM: the smaller CIFAR-10 non-IID client pool was chosen to keep the number of samples

per client, and hence the per-client Dirichlet partition, large enough to be meaningful for ResNet18/ShuffleNetV2 training under $\alpha = 1$; please confirm or replace this rationale with the actual reason, and consider adding a matched 100-client non-IID run if feasible, to make the CIFAR-10 IID/non-IID comparison directly comparable, as reviewers requested.] All settings assume 40% malicious clients, the theoretical upper bound for MKRUM [10] performing constant label-flipping attacks. The trust update rule (Section IV, M2) used learning rate $\eta = [VALUE]$, trust threshold scale $k = [VALUE]$ (i.e., $\tau_{thresh} = 1/(kN)$), and flag threshold $cnt_{max} = [VALUE]$ rounds; [AUTHOR TO INSERT: the actual values used to produce Table I, and, if available even a coarse sensitivity sweep over one or two of these, e.g., $\eta \in \{0.05, 0.1, 0.2\}$, as several reviewers specifically requested these settings and a sensitivity discussion.]

antiflipper_combined_subplots_trust_updated

Fig. 2: Average Client Trust Over First 25 Rounds

A. Evaluation Metrics

We evaluated using three metrics: Overall Accuracy (All-Acc), Test Error (TE), and Aggregation Time. All-Acc measures prediction correctness; TE (from cross-entropy loss) reflects robustness against label-flipping; Aggregation Time measures server-side efficiency, essential for scalability.

B. Malicious Node Detection

AntiFlipper distinguishes malicious from honest clients using the global model’s accuracy on local datasets. When a client flips labels, its local accuracy drops because the global model, trained on honest data, performs poorly on manipulated data. Malicious accuracy declines rapidly, near zero early on MNIST and within 20 rounds on CIFAR, while honest nodes maintain high accuracy. Figure 2 shows trust scores diverging clearly between honest and malicious clients within a few rounds.

C. Experimental Results

On MNIST (Table I), AntiFlipper, LFighter, Tolpegin, MKRUM, FLAME, TMean, and Median all achieved high

¹The source code is available at: <https://github.com/abidh8820/AntiFlipper>

TABLE I: Comparison of Different Methods on MNIST and CIFAR-10 (IID and Non-IID)

	Methods	FoolsGold	FedAvg	Median	TMean	FLAME	MKRUM	LFighter	Tolpegin	AntiFLipper
MNIST (IID)	Global Accuracy (%)	0.36	51.85	96.10	96.18	97.44	97.45	97.45	97.45	97.06
	Test Error	17.60	0.86	0.18	0.18	0.09	0.09	0.09	0.09	0.10
	Aggregation Time (ms)	9.36	4.61	12.03	12.86	45.61	15.55	119.65	160.94	3.01
MNIST (NON-IID)	Global Accuracy (%)	3.58	53.99	95.67	95.61	97.03	96.74	96.76	96.80	96.77
	Test Error	6.72	0.76	0.22	0.23	0.11	0.12	0.13	0.13	0.11
	Aggregation Time (ms)	8.65	4.44	13.09	11.73	45.66	13.70	119.37	168.88	2.96
CIFAR (IID)	Global Accuracy (%)	70.13	47.21	63.90	63.77	67.30	32.86	69.96	70.28	70.09
	Test Error	1.10	1.43	1.10	1.12	1.15	2.31	1.12	1.11	1.10
	Aggregation Time (ms)	53.87	50.76	660.39	641.66	1489.91	686.93	468.02	354.96	48.01
CIFAR (NON-IID)	Global Accuracy (%)	56.01	50.95	60.93	59.66	37.77	44.08	64.58	53.58	68.23
	Test Error	1.12	1.31	1.18	1.19	3.23	1.70	1.21	1.31	1.14
	Aggregation Time (ms)	53.16	49.75	712.13	678.38	1334.06	727.38	449.12	339.47	48.43

accuracy. FoolsGold collapsed in non-IID settings (3.58%), misidentifying heterogeneous honest gradients as sybils. FedAvg amplified poisoned updates. AntiFLipper matched top defenses in accuracy while achieving the lowest aggregation time (3.01 ms IID; 2.96 ms non-IID), 53 times faster than LFighter and 15 times faster than FLAME. AntiFLipper’s aggregation time (3.01 ms) is also marginally lower than the undefended FedAvg baseline (4.61 ms) despite performing conceptually more work (a trust-weighted sum versus a uniform average). We attribute this to implementation-level rather than algorithmic factors: our FedAvg baseline recomputes uniform weights and iterates over the full client list, including clients on the malicious list β , on every round, whereas AntiFLipper’s aggregation loop (Algorithm 1, line 7) iterates only over the already-filtered honest subset $C \setminus \beta$, which shrinks over rounds as malicious clients are excluded. At the millisecond scale reported here, both figures are also within the range of ordinary system timing jitter (CPU scheduling, memory allocation), so this difference should be read as “AntiFLipper’s overhead relative to uniform averaging is negligible” rather than as evidence that trust-weighting is algorithmically faster than averaging; the meaningful comparison is against the two to three orders of magnitude slower FLAME and LFighter.

On CIFAR-10, results followed similar trends with generally lower accuracy due to complexity. In IID, AntiFLipper (70.09%) matched LFighter, Tolpegin, and FoolsGold in terms of accuracy at the lowest aggregation time (48 ms vs. 1490 ms for FLAME). In non-IID, AntiFLipper led in both accuracy (68.23%) and efficiency, outperforming all methods including LFighter (64.58%) and Tolpegin (53.58%). These CIFAR-10 non-IID results are particularly significant as they confirm that AntiFLipper’s relative-deviation approach generalizes to complex datasets and heterogeneous distributions without requiring distribution-specific assumptions or server-side reference data, in contrast to methods such as [17], [16] that were only validated on MNIST.

We further tested AntiFLipper under dynamic attack scenarios: (1) attacks starting after $n/2$ rounds (with n the total rounds), and (2) poisoning every n rounds ($n = 15$). As shown in Table II, AntiFLipper maintained accuracies within $\pm 0.5\%$ of baseline and detected all malicious nodes. Delayed

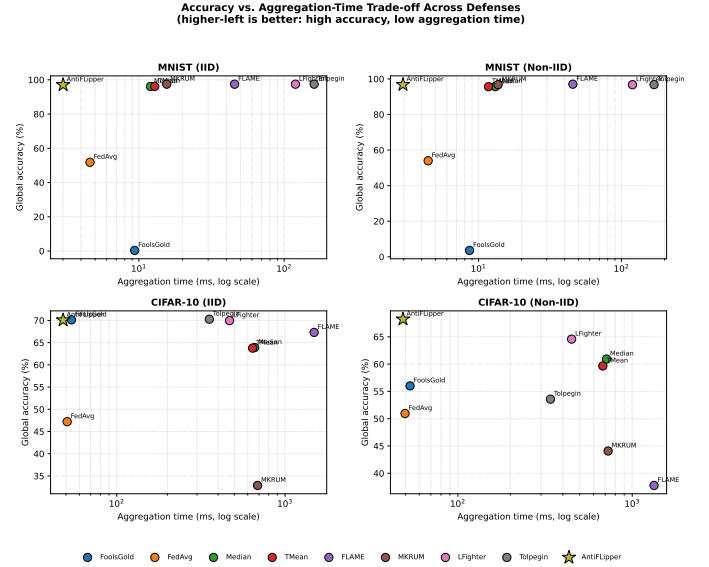


Fig. 3: Accuracy vs. aggregation-time trade-off across all nine methods and four dataset/distribution settings (data as reported in Table I; note the log-scaled time axis). AntiFLipper (star) consistently occupies the upper-left region, matching the accuracy of the best-performing baselines while incurring the lowest aggregation time in every setting.

or intermittent attacks were easier to detect, as the model, already dominated by benign updates, remained stable and deviations were more apparent. In both cases, AntiFLipper remained robust, efficient, and incurred similar aggregation time as before.

TABLE II: Global test accuracies (%) under dynamic attack scenarios

Scenario	MNIST		CIFAR-10	
	IID	Non-IID	IID	Non-IID
Scenario 1	96.93	97.03	72.66	70.62
Scenario 2	96.68	96.82	72.59	70.59

Figure 3 restates the results of Table I as an accuracy-versus-aggregation-time trade-off, which makes the practical value of AntiFLipper’s design more directly visible than the raw table: across all four dataset/distribution settings, AntiFLipper sits in

the upper-left region (high accuracy, low time), whereas the next most accurate defenses (LFighter, FLAME, Tolpegin) are shifted one to two orders of magnitude to the right on the log-scaled time axis.

In summary, AntiFLipper delivers robust, efficient, and scalable defense across datasets, distributions, and dynamic attack conditions.

VI. SECURITY ANALYSIS

In this section, we formalize AntiFLipper’s trust-based defense against label-flipping attacks. Our approach quantifies deviations in local model accuracy to adjust client trust scores, enabling the system to progressively isolate and exclude malicious participants. This establishes AntiFLipper as an efficient and robust defense for secure federated learning in adversarial federated learning environments.

A. Accuracy-Based Detection of Label Flipping Behavior

A key challenge in federated learning is mitigating label-flipping attacks, where malicious clients poison training by mislabeling data. AntiFLipper tackles this by exploiting the accuracy drop observed when the global model is trained on such poisoned data.

Lemma-1: *In AntiFLipper, if a client c_i flips labels in its local dataset d_i , converting it into a poisoned distribution d'_i , then the global model M yields lower accuracy on d'_i than on honest data, exposing adversarial behavior.*

Proof: Let $C = \{c_1, c_2, \dots, c_n\}$ be the set of clients participating in federated learning, and let M be the global model aggregated from local models of the majority of honest clients. If a client $c_i \in C$ performs label-flipping, its local data distribution becomes d'_i , which diverges from the honest distributions d_j , where $j \neq i$. The global model M minimizes loss over honest client distributions d_j , so when evaluated on a flipped distribution d'_i , it incurs higher loss: $L(M, d'_i) > L(M, d_j)$. Since accuracy decreases as loss increases, this implies: $\text{acc}(M, d'_i) < \text{acc}(M, d_j)$. To detect deviation, AntiFLipper compares each client’s accuracy to the average acc_{avg} . For a malicious client c_i , $\text{acc}_{\text{avg}} - \text{acc}(M, d'_i) > \text{acc}_{\text{avg}} - \text{acc}(M, d_j)$, and squaring amplifies this gap: $\Delta_i = (\text{acc}_{\text{avg}} - \text{acc}(M, d'_i))^2$. This deviation is used to reduce the client’s trust score, allowing the system to gradually identify and isolate clients that consistently behave maliciously. **Transient non-IID deviations are handled by the flag-counter mechanism (M4), which requires persistent low trust before exclusion, preventing removal of honest clients with skewed data.**

B. Trust Score Convergence for Malicious Exclusion

Lemma-2: *In AntiFLipper, a client c_i is considered malicious and excluded from training if its trust score satisfies $\tau_i < \tau_{\text{threshold}}$ for more than x rounds, indicating consistent behavior associated with label-flipping attacks.*

Proof: Each client $c_i \in C$ starts with a trust value $\tau_i = \frac{1}{n}$, ensuring $\sum_{i=1}^n \tau_i = 1$. These values are used to weight local models in global aggregation as $M \leftarrow M + w_i \cdot m_i$, where $w_i =$

$\frac{\tau_i}{W}$, $W = \sum \tau_i = 1$. After each round, trust is updated based on the squared deviation from the average accuracy: $\tau_i \leftarrow \tau_i \pm \eta \cdot (\text{acc}_i - \text{acc}_{\text{avg}})^2$, and then normalized to maintain $\sum \tau_i = 1$. Since trust updates are applied independently per client, individual scores fluctuate while the total remains constant. A client is flagged if $\tau_i < \tau_{\text{threshold}} = 1/kn$, where $k > 1$, indicating its influence has dropped by a factor of k . Given $\tau_i \ll \tau_{\text{avg}} = 1/n$, its contribution becomes negligible. If this persists for x rounds, the client is considered malicious and excluded from further training.

C. Dependence on the Malicious Client Ratio

Lemma-1’s guarantee that $\text{acc}(M, d'_i) < \text{acc}(M, d_j)$ rests on the assumption that M is aggregated predominantly from honest updates, so that M ’s decision boundaries reflect the honest label distribution rather than the flipped one. Let $p \in [0, 1]$ denote the fraction of participating clients that are malicious in a given round, and let acc_H and $\text{acc}_{M'}$ denote the global model’s expected accuracy on honest and flipped data, respectively, before any exclusion. Because acc_{avg} is itself a weighted mixture of both populations, $\text{acc}_{\text{avg}} \approx (1-p) \cdot \text{acc}_H + p \cdot \text{acc}_{M'}$, so the detectable gap for a malicious client shrinks as

$$\text{acc}_{\text{avg}} - \text{acc}_{M'} \approx (1-p)(\text{acc}_H - \text{acc}_{M'}),$$

which is monotonically decreasing in p and vanishes as $p \rightarrow 1$. In the limit where malicious clients form a majority ($p > 0.5$), M itself is increasingly shaped by poisoned updates each round, so $\text{acc}_H - \text{acc}_{M'}$ collapses and the accuracy-deviation signal that Lemma-1 relies on degrades along with it, a fundamental limitation shared by any detector that infers trust purely from relative deviation rather than absolute, distribution-independent ground truth. This is consistent with why our experiments, and the broader label-flipping literature [6], [14], evaluate at a fixed 40% malicious ratio, the standard worst case tolerated by comparably-scoped defenses such as Multi-Krum [10], rather than the majority-adversary regime, which is generally considered out of scope for detection-based (as opposed to certified-robust) defenses. We have not empirically swept p across the full $[0, 1]$ range in this version of the paper; Section VIII discusses this as a direction for follow-up work, including deriving the precise threshold p^* at which detection accuracy degrades below a target recall.

D. Robustness to Non-IID Skew and Adaptive Adversaries

The flag-counter mechanism (M4) is the primary safeguard against penalizing honest clients whose accuracy is transiently low due to non-IID skew rather than malicious behavior: a single low-accuracy round only increments cnt_j , and exclusion requires cnt_{max} consecutive (or cumulative, depending on configuration) violations. This tolerates the natural round-to-round accuracy fluctuation that heterogeneous partitions produce. However, as Section II-B notes, an honest client whose local distribution is persistently and severely skewed, rather than merely transiently divergent, could in principle be misclassified if its accuracy stays below τ_{thresh} for cnt_{max} rounds purely due to heterogeneity rather than poisoning. Our current experiments

use a single, moderate heterogeneity setting (Dirichlet $\alpha = 1$); we have not stress-tested this false-positive failure mode under more extreme skew (e.g., $\alpha = 0.1$), and we report this explicitly as a limitation in Section VIII rather than claiming coverage we have not evaluated.

E. Privacy Considerations of Accuracy Reporting

While AntiFLipper avoids transmitting raw data or gradients, the scalar acc_{ij} is not information-free: repeated accuracy reports across rounds implicitly reveal how well the evolving global model fits a client’s local distribution, which could in principle support membership-inference-style reasoning about a client’s data composition, particularly for clients with small or highly unique local datasets. This risk is qualitatively smaller than gradient- or update-based leakage (the object exploited by classical FL membership-inference attacks), since a single scalar carries far less information than a full model update, but it is not zero. A standard mitigation would be to inject calibrated noise into acc_{ij} before transmission, e.g., via a Laplace or Gaussian mechanism sized to a target (ϵ, δ) -differential-privacy budget, analogous to the noise-addition step used by FLAME [8] for its clustering inputs. Because the trust update (Section IV, M2) already operates on a *squared deviation* from a population average rather than the raw value, we expect it to be reasonably tolerant of small additive noise, but quantifying the resulting privacy-utility trade-off (i.e., how much noise the detector can absorb before its recall degrades) requires a dedicated empirical study that we leave to future work rather than claim here.

F. Convergence Under Progressive Client Exclusion

A natural concern is whether progressively excluding clients (M4) and re-weighting the remainder (M5) preserves the convergence behavior of the underlying federated averaging procedure. Since M5 performs a convex combination of surviving client models with weights $w_j = \tau_j/W$ summing to one, each round of AntiFLipper is structurally a weighted instance of FedAvg restricted to the current honest set $C \setminus \beta$. Existing convergence analyses of weighted FedAvg under bounded client heterogeneity [29] therefore apply directly to any *fixed* client set, and because β is monotonically non-decreasing and, by Lemma-2, only clients whose behavior is persistently and predominantly malicious are added to β , the honest sub-population’s data heterogeneity bound is not made worse by exclusion. This gives an informal argument that AntiFLipper inherits the convergence guarantees of weighted FedAvg on the (shrinking) honest client set, modulo the standard caveat that these guarantees assume bounded gradient dissimilarity across the surviving honest clients. We view a formal, non-asymptotic convergence proof that explicitly accounts for the round-varying client set $C \setminus \beta_i$ as valuable future work; our empirical results (Table I, Table II) provide supporting evidence of stable convergence but do not constitute a proof.

VII. COMPUTATIONAL AND COMMUNICATION OVERHEAD OF ANTIFLIPPER

To quantify the computational overhead introduced by local evaluation in AntiFLipper, we consider a neural network model $f(x; w)$ with W parameters, trained on each client k with dataset D_k of n_k samples. Clients perform E local SGD epochs (batch size B), where the per-sample forward and backward pass costs are C_f and $C_b \approx 2C_f$, respectively. Thus, the training cost per client per round is $C_{\text{train}} \approx 3E \cdot n_k \cdot C_f$.

Local evaluation on a fraction γ of the data adds $C_{\text{eval}} = \gamma \cdot n_k \cdot C_f$, leading to a total cost of $C_{\text{total}} = n_k \cdot (3E + \gamma) \cdot C_f$, and relative overhead $\frac{\gamma}{3E}$. In our experiments with $E = 3$, full evaluation ($\gamma = 1$) yields a worst-case overhead of 11.1%. The global accuracies reported in Table I correspond to this full local data evaluation setting.

To reduce cost, we performed evaluation on only 10% of local data ($\gamma = 0.1$), resulting in just 1.1% overhead. Despite this, the system achieves strong performance: 97.11% (MNIST IID), 96.79% (MNIST Non-IID), 74.81% (CIFAR-10 IID), and 68.18% (CIFAR-10 Non-IID), with early malicious client detection, no increase in aggregation time, and no noticeable drop in accuracy—sometimes even slight improvements.

AntiFLipper avoids server-side inspection of updates, eliminating the main overhead in competing methods. Instead of FLAME-style aggregation, it shifts evaluation to clients in parallel, keeping aggregation lightweight and scalable. The parameter γ provides a flexible trade-off between efficiency and detection sensitivity.

A. Communication Overhead

Beyond computation, we analyze communication cost, which was not previously discussed. In each round, every participating client downloads the global model (W parameters) and uploads its locally trained model update (also W parameters), exactly as in vanilla FedAvg [1]; AntiFLipper adds only the scalar acc_{ij} (a single 32-bit float, 4 bytes) to the upload. For a model with W parameters transmitted at 32-bit precision, the relative communication overhead is $4/(4W) = 1/W$, i.e., asymptotically negligible for any model beyond a few parameters (e.g., $< 0.001\%$ for a CNN with $W \approx 4 \times 10^5$ parameters, as used for MNIST). This is in sharp contrast to methods that require additional communication rounds or payload expansion: cryptographic defenses such as Shen et al. [16] transmit homomorphically encrypted updates, which typically expand payload size by one to two orders of magnitude, and server-reference-based methods such as FLTrust [11] and HSCS [17] require the server (not clients) to additionally hold and periodically refresh a labeled reference dataset. FLAME [8] and clustering-based defenses do not add client-side communication either, but their overhead, as quantified in Table I, is concentrated in server-side computation over the received updates rather than in the wire cost, which is precisely the overhead AntiFLipper is designed to avoid.

VIII. LIMITATIONS AND THREATS TO VALIDITY

We deliberately scope this paper to a computationally efficient, client-side detection mechanism for the specific and widely-studied label-flipping threat model, evaluated with no additional experimentation beyond what was already run for the original submission. In the interest of transparency, we summarize here the evaluation gaps identified during review that would require new experiments to close, rather than overstating the paper’s current coverage:

- **Statistical significance.** All reported accuracies and aggregation times (Table I, Table II) come from single runs rather than an average over multiple random seeds, so we cannot currently report standard deviations or confidence intervals, or certify that the small accuracy gaps between AntiFLipper and the closest baselines (e.g., LFighter, Tolpegin) are statistically significant rather than run-to-run noise.
- **Malicious ratio sweep.** Consistent with Section VI-C, all results use a fixed 40% malicious client ratio; we have not empirically swept lower or higher ratios (e.g., 10–50%) to trace the detection-performance curve or empirically locate the failure threshold p^* .
- **Heterogeneity range.** All non-IID results use a single, moderate Dirichlet setting ($\alpha = 1$); more extreme heterogeneity (e.g., $\alpha = 0.1$) is not evaluated, so the false-positive risk discussed in Section VI for severely skewed honest clients is not empirically bounded.
- **Baseline scope.** Our empirical comparison targets defenses specifically designed for, or commonly benchmarked against, label-flipping (FoolsGold, FLAME, LFighter, Tolpegin) plus foundational robust-aggregation baselines (Median, Trimmed Mean, Multi-Krum). We have not benchmarked against Bulyan [15], CRFL [23], or FLTrust [11], which target a broader Byzantine/backdoor threat model; a fair comparison would require re-tuning these methods for the label-flipping setting and is left to future work.
- **Domain and architecture coverage.** Evaluation is restricted to two image-classification benchmarks (MNIST, CIFAR-10) with CNN/ResNet/ShuffleNet architectures. Tabular, time-series, and NLP workloads, which are also relevant to the IoT and edge scenarios this paper motivates, are not evaluated.
- **Hyperparameter sensitivity.** We report the trust threshold scale k , flag threshold $cnt_{\max}$, trust learning rate η , and evaluation fraction γ as tunable parameters (Section IV), and provide an analytical trade-off for γ (Section VII), but have not run a controlled sensitivity sweep over k , $cnt_{\max}$, and η .
- **Deployment realism.** All experiments assume synchronous, fully reliable client participation each round. Client dropout, stragglers, communication failures, and partial-participation schedules, all common in real IoT/edge deployments, are not modeled.
- **Resource-constrained hardware.** Our overhead analysis

(Section VII) is analytical and our timing measurements (Table I) were collected on a workstation-class GPU server; we have not measured memory, energy, or latency on representative constrained edge/IoT hardware (e.g., Raspberry Pi- or microcontroller-class devices), so the suitability claims for such deployments should be read as motivated by the analytical overhead formula rather than validated on-device.

We view each of these as a well-defined, tractable follow-up rather than a fundamental obstacle to the approach, and we have restructured Sections II-B and VI in this revision to analytically characterize several of these gaps (the malicious-ratio threshold, adaptive/colluding adversaries, and privacy leakage) even where we could not close them empirically without new experimentation.

IX. CONCLUSION

In this paper, we introduced AntiFLipper, a lightweight and effective defense mechanism for detecting and mitigating label-flipping attacks in federated learning. Unlike existing methods that trade off security for efficiency, AntiFLipper achieves a strong balance between both. Through extensive evaluations on IID and non-IID data distributions across two benchmark datasets, we show that AntiFLipper consistently matches or exceeds the accuracy of state-of-the-art defenses. Remarkably, it achieves significantly lower aggregation time—even outperforming the baseline FedAvg in some cases. This optimal trade-off makes AntiFLipper particularly suitable for real-world FL deployments in privacy-sensitive, time-constrained [edge and IoT network](#) environments where both security and efficiency are non-negotiable.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.
- [2] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, “Federated learning: Challenges, methods, and future directions,” *IEEE signal processing magazine*, vol. 37, no. 3, pp. 50–60, 2020.
- [3] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, “Advances and open problems in federated learning,” *Foundations and trends® in machine learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [4] L. Lyu, H. Yu, and Q. Yang, “Threats to federated learning: A survey,” *arXiv preprint arXiv:2003.02133*, 2020.
- [5] B. Biggio, B. Nelson, and P. Laskov, “Support vector machines under adversarial label noise,” in *Asian conference on machine learning*. PMLR, 2011, pp. 97–112.
- [6] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, “Data poisoning attacks against federated learning systems,” in *Computer security—ESORICS 2020: 25th European symposium on research in computer security, ESORICS 2020, guildford, UK, September 14–18, 2020, proceedings, part i 25*. Springer, 2020, pp. 480–501.
- [7] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, “How to backdoor federated learning,” in *International conference on artificial intelligence and statistics*. PMLR, 2020, pp. 2938–2948.
- [8] T. D. Nguyen, P. Rieger, R. De Viti, H. Chen, B. B. Brandenburg, H. Yalame, H. Möllering, H. Fereidooni, S. Marchal, M. Miettinen *et al.*, “{FLAME}: Taming backdoors in federated learning,” in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 1415–1432.
- [9] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, “Byzantine-robust distributed learning: Towards optimal statistical rates,” in *International conference on machine learning*. Pmlr, 2018, pp. 5650–5659.

- [10] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," *Advances in neural information processing systems*, vol. 30, 2017.
- [11] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "Fltrust: Byzantine-robust federated learning via trust bootstrapping," in *Network and Distributed System Security Symposium (NDSS)*, 2021.
- [12] C. Fung, C. J. Yoon, and I. Beschastnikh, "The limitations of federated learning in sybil settings," in *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, 2020, pp. 301–316.
- [13] H. Wang, K. Sreenivasan, S. Rajput, H. Vishwakarma, S. Agarwal, J.-y. Sohn, K. Lee, and D. Papailiopoulos, "Attack of the tails: Yes, you really can backdoor federated learning," *Advances in neural information processing systems*, vol. 33, pp. 16 070–16 084, 2020.
- [14] N. M. Jebreel, J. Domingo-Ferrer, D. Sánchez, and A. Blanco-Justicia, "Lfighter: Defending against the label-flipping attack in federated learning," *Neural Networks*, vol. 170, pp. 111–126, 2024.
- [15] E. M. El Mhamdi, R. Guerraoui, and S. Rouault, "The hidden vulnerability of distributed learning in Byzantium," in *International Conference on Machine Learning (ICML)*. PMLR, 2018, pp. 3521–3530.
- [16] X. Shen, Y. Liu, F. Li, and C. Li, "Privacy-preserving federated learning against label-flipping attacks on non-iid data," *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1241–1255, 2024.
- [17] Y. Li, H. Chen, W. Bao, Z. Xu, and D. Yuan, "Honest score client selection scheme: Preventing federated learning label flipping attacks in non-iid scenarios," *arXiv preprint arXiv:2311.05826*, 2023. [Online]. Available: <https://arxiv.org/abs/2311.05826>
- [18] Y. Jiang, W. Zhang, and Y. Chen, "Data quality detection mechanism against label flipping attacks in federated learning," in *IEEE Transactions on Information Forensics and Security*, vol. 18. IEEE, 2023, pp. 1625–1637.
- [19] P. R. Ovi, A. Gangopadhyay, R. F. Erbacher, and C. Busart, "Confident federated learning to tackle label flipped data poisoning attacks," in *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications V, SPIE*, vol. 12538, 2023.
- [20] H. Li, Z. Shi, M. Jin, A. Yin, and Z. Zhao, "FLGT: Label-flipping-robust federated learning via guiding trust," *Knowledge and Information Systems*, pp. 1–24, 2025.
- [21] Anonymous, "FedAlign: Defense against label flipping attacks in federated learning via dual-granularity distribution alignment," *Neurocomputing*, 2026, in press, ScienceDirect early access.
- [22] H. Abroshan, "AI to protect AI: A modular pipeline for detecting label-flipping poisoning attacks," *Machine Learning with Applications*, vol. 22, p. 100768, 2025.
- [23] C. Xie, M. Chen, P.-Y. Chen, and B. Li, "CRFL: Certifiably robust federated learning against backdoor attacks," in *International Conference on Machine Learning (ICML)*. PMLR, 2021, pp. 11 372–11 382.
- [24] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [25] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.
- [26] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [27] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "Shufflenet v2: Practical guidelines for efficient cnn architecture design," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 116–131.
- [28] T. S. Ferguson, "A bayesian analysis of some nonparametric problems," *The annals of statistics*, pp. 209–230, 1973.
- [29] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-IID data," in *International Conference on Learning Representations (ICLR)*, 2020.